

Das RSA-Verfahren

DC-03, Kap. 3

3.1	RSA-Schlüsselpaare	41
3.2	Das RSA-Verschlüsselungsverfahren	45
3.3	Das RSA-Signaturverfahren	51

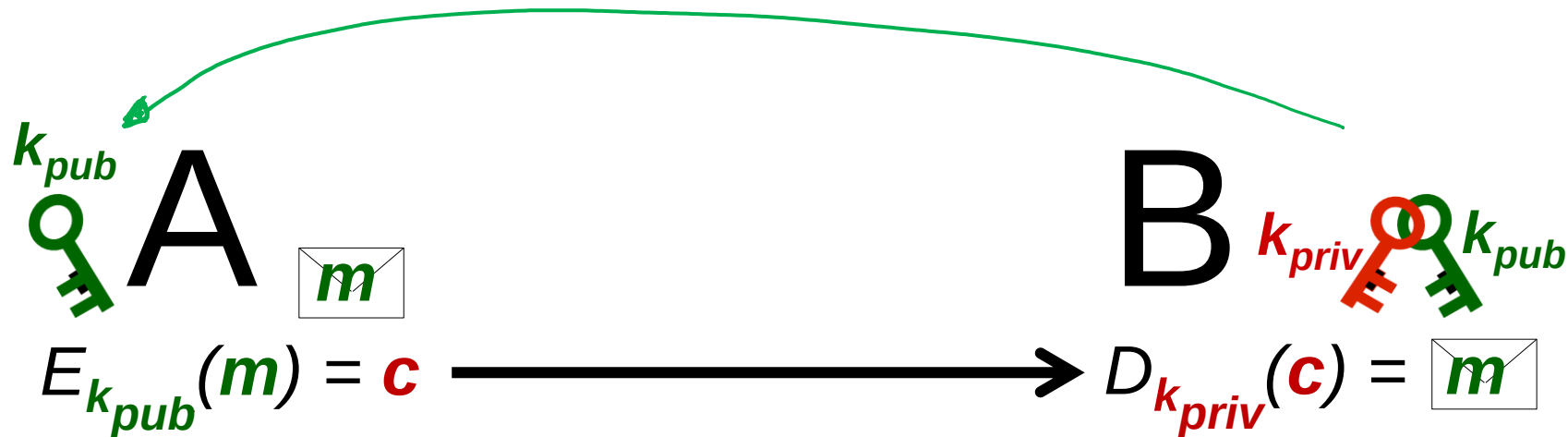
Asymmetrische Verschlüsselungsverfahren

Es gibt eine Funktionen $d_{\mathcal{K}}$ zur Ableitung *öffentlicher Schlüssel* und eine Funktion E_{pub} zur Verschlüsselung beliebiger Nachrichten mit den öffentlichen Schlüsseln:

$$d_{\mathcal{K}} : \mathcal{K} \rightarrow \mathcal{K}_{pub}, \quad k \mapsto k_{pub}$$

$$E_{pub} : \mathcal{K}_{pub} \times \mathcal{M} \rightarrow \mathcal{M}, \quad k \mapsto k_{pub}$$

$$E_k(m) = E_{pub}(k_{pub}, m) =: E_{k_{pub}}(m) \quad \text{für alle } k \in \mathcal{K}, m \in \mathcal{M}$$





Wdh.:

Satz 1.6 (Kleiner Satz von Fermat)

Ist p eine Primzahl, so gilt für alle $a \in \mathbb{Z}$ mit $\text{ggT}(a, p) = 1$:

$$a^{p-1} \equiv 1 \pmod{p}$$

$$\Leftrightarrow a^p \equiv a \pmod{p} \quad (\text{stimmt immer!})$$

$$\Leftrightarrow a^{p-1} = 1 \text{ in } \mathbb{Z}_p \quad \text{f. a. } a \neq 0 \quad (\Leftrightarrow \text{f. a. } a \in \mathbb{Z}_p^*)$$

$$\left[\Leftrightarrow a^p \equiv a \text{ in } \mathbb{Z}_p \quad \text{f. a. } a \in \mathbb{Z}_p \right]$$

$$\Leftrightarrow o(a) \mid (p-1) \quad \text{f. a. } a \in \mathbb{Z}_p^*$$

Satz 3.1

Es seien p und q zwei verschiedene Primzahlen und:

$$n := p \cdot q$$

Weiterhin sei $r \in \mathbb{Z}$ mit

$$r \equiv 1 \pmod{p-1} \quad (3.1)$$

und:

$$r \equiv 1 \pmod{q-1} \quad (3.2)$$

Dies gilt zum Beispiel für $r = 1 + (p-1)(q-1)$ oder $r = 1 + \text{kgV}(p-1, q-1)$.

Dann gilt für alle $z \in \mathbb{Z}_n$:

$$z^r = z \quad (3.3)$$

$$\begin{aligned} & \left. \begin{aligned} (p-1) & \mid (r-1) \\ (q-1) & \mid (r-1) \end{aligned} \right\} \Leftrightarrow (x) \\ & x) \text{ kgV}(p-1, q-1) \mid r-1 \end{aligned}$$

$$\begin{aligned} & \text{in } \mathbb{Z}_p \\ & z^r = z \cdot z^{r-1} = z \cdot z^{(p-1) \vee} = \begin{cases} 0 & z=0 \\ z \cdot 1^v = z & \text{für } z \in \mathbb{Z}_p^* \end{cases} \end{aligned}$$

$$\text{D.h. } p \mid (z^r - z), \text{ genauso gilt } q \mid (z^r - z) \Rightarrow \begin{matrix} p \neq q \\ p \mid q \end{matrix} p \cdot q \mid z^r - z \Leftrightarrow (3.3)$$

3.1 RSA-Schlüsselpaare

RSA-Schlüsselpaar

Alice generiert ihr Schlüsselpaar folgendermaßen:

- Alice bestimmt zwei verschiedene Primzahlen

$$p \quad \text{und} \quad q \quad \text{mit} \quad e \nmid (p-1), e \nmid (q-1)$$

Probabilistische
Primzahltests

- Alice berechnet:

$$n = p \cdot q$$

privat
"Wurzel"-Secret

- Alice berechnet

$$v = \text{kgV}(p-1, q-1)$$

$\bar{E}_A$

- Alice wählt eine Zahl:

$$e \in \mathbb{Z}_v^*$$

typisch $e = 2^{16} + 1$ P_e
 $\Rightarrow \text{ggT}(e, v) = 1$

$$k_{A, \text{pub}} = (n, e)$$



- Alice bestimmt die Inverse zu e in $\mathbb{Z}_v$:

$$d = e^{-1} \in \mathbb{Z}_v^*$$

$\bar{E}_A$

$$k_{A, \text{priv}} = (n, d)$$



Beispiel 3.1

```

n = 16168071105410691789333727253545285425054602153245
90204154619291826180517268289289260262961391645385
87252292573042195868358338780212272813328285432930
98938867283088253785506450582562311172139885613039
47832745852869850074523674809445697461898848258982
56177917054178929148242979069451204216293772563056
69543996313975448783597681650671949525006213463201
70488387186763666227169881155057139124481245006496
90477201599126274576409287155040493852180020698941
33852393880073215037370387712463302102903103361251
39476232741977840751451359061623686793523945503881
07459238974533011939826218652756330708024526384519
82798725355045671
  
```

```
e = 65537
```

```

d = 12935048967741785432631124210108586072088482843263
94457851824140699609333375207043570717420566805787
14417996005153553164926139721441168929740889296419
39111687892135485641418957488821690028192295687365
70071357104500846531080539506063091220566709063810
88055915696243490136879562362779277957012299663185
50826720804422302159550428738857532165935936434657
09409990352714686398188649999443761602361578163048
17343088468413646273406481388390630608925889022555
01537512298374597901048595576433506634785244042404
20343235304253602775533150811070033763661398799934
67055475814167460324061409325201045135140539718680
44161275601117057
  
```

```

KeyPairGenerator kpgenerator
    = KeyPairGenerator.getInstance("RSA");
kpgenerator.initialize(2048);
KeyPair rsaKeyPair = kpgenerator.generateKeyPair();
  
```

```

RSAPublicKey publicKey
    = (RSAPublicKey) rsaKeyPair.getPublic();
RSAPrivateKey privateKey
    = (RSAPrivateKey) rsaKeyPair.getPrivate();
  
```

```

BigInteger n = publicKey.getModulus();
BigInteger e = publicKey.getPublicExponent();
BigInteger d = privateKey.getPrivateExponent();
  
```

Beispiel 3.2 Schulbuch - Beispiel

$$p = 41 \quad \text{und} \quad q = 43$$

$$n = p \cdot q = 1763$$

$$v = \frac{40 \cdot 42}{2} = 840$$

kleinste mögliche e : $e = 11$

$v = 840$	11		9
1	0	840	
0	1	11	76
1	-76	4	2
-2	153	3	1
3	-229	1	

$$d = 840 - 229 = 611$$

Beispiel 3.2

$$p = 41 \quad \text{und} \quad q = 43$$

$$n = p \cdot q = 1763$$

$$k_{A, \text{pub}} = (n, e) = (1763, 11)$$

$$k_{A, \text{priv}} = (n, d) = (1763, 611)$$

Frage 16

Gegeben seien die Zahlen $p, q \in \mathbb{N}$ mit den Bitlängen $l_2(p)$ und $l_2(q)$. Welche Bitlänge hat das Produkt pq ?

kleinste Zahlen mit $l_2(n) = 1024$: $p = 2^{1023}$

$$p = q : p \cdot q = 2^{2046}, \quad l_2(p \cdot q) = 2047$$

Ansatz: $p = 2^{1023} + 2^{1022} \Rightarrow l_2(p \cdot p) = 2048$

Frage: Wie viele Primzahlen in $[2^{1023} + 2^{1022}, \dots, 2^{1024} - 1] = I$

$|I| = 2^{1022}$, Anzahl der P_2 in $I \approx \pi(2|I|) - \pi(|I|)$ (Primzahlssatz)

$$\begin{aligned} &\approx \frac{2 \cdot |I|}{\log(2 \cdot |I|)} - \frac{|I|}{\log(|I|)} \\ &= \frac{2^{1023}}{1023 \cdot \log(2)} - \frac{2^{1022}}{1022 \cdot \log(2)} = \\ &\approx 6,33 \cdot 10^{304} \end{aligned}$$

NB: $\log_a(x) = \log_a(b^{\log_b(x)}) = \log_b(x) \cdot \log_a(b)$

Frage 17

Für die Bitlängen der Zahlen $p, q \in \mathbb{N}$ gelte:

$$l = l_2(p) = l_2(q)$$

Im Dualsystem besitze p die folgende Darstellung:

$$p = 1 \underbrace{0 \dots 0}_m 1 * \dots *$$

D. h., in der Binärdarstellung von p treten mindestens zwei Stellen mit dem Wert 1 auf und zwischen den beiden höchstwertigen solchen Stellen liegen m Stellen, deren Wert 0 ist ($m \in \mathbb{N}_0$). Die Ziffern, die den ersten $m+2$ Ziffern folgen, sind beliebig.

Weiterhin seien in der Binärdarstellung von q die $m+2$ höchstwertigen Bits alle 1:

$$q = \underbrace{1 \dots 1}_{m+2} * \dots *$$

Zeigen Sie, dass unter diesen Voraussetzungen gilt:

$$l_2(pq) = 2l$$

Schreiben Sie ein Programm zur Erzeugung von RSA-Schlüsselpaaren. Messen Sie wiederholt (mindestens 10 mal) die Laufzeiten für die Generierung von Schlüsselpaaren mit einer Schlüssellänge von 1024, 2048 und 4096 Bit.

- (i) Verwenden Sie lediglich die Klasse `BigInteger` und die durch sie bereitgestellten Methoden.
- (ii) Verwenden Sie die Klassen `KeyPairGenerator` und `RSAKeyGenParameterSpec`.

```

16 //Performance Test for RSA Key Generation
17 public static void rsa_keygen_performance(int bitlength, int noOfTests)
18     throws NoSuchAlgorithmException, InvalidAlgorithmParameterException, InterruptedException {
19
20     RSAKeyGenParameterSpec rsaKeyParameter
21         = new RSAKeyGenParameterSpec(bitlength, RSAKeyGenParameterSpec.F4);
22
23     KeyPairGenerator kpgenerator = KeyPairGenerator.getInstance("RSA");
24     kpgenerator.initialize(rsaKeyParameter);
25
26     KeyPair rsaKeyPair = null;
27
28     System.out.println("Generating " + noOfTests + " RSA-Keypairs of length "
29         + bitlength + " bits:");
30     for (int i = 1; i <= noOfTests; ++i) {
31         long t = System.nanoTime();
32         rsaKeyPair = kpgenerator.generateKeyPair();
33         t = System.nanoTime() - t;
34         System.out.println(
35             "Time to generate a RSA key pair ("
36             + String.format("%2d", i) + "):"
37             + String.format("%10.1f", (double) t / 1000000));
38     }
39     System.out.println();
40 }

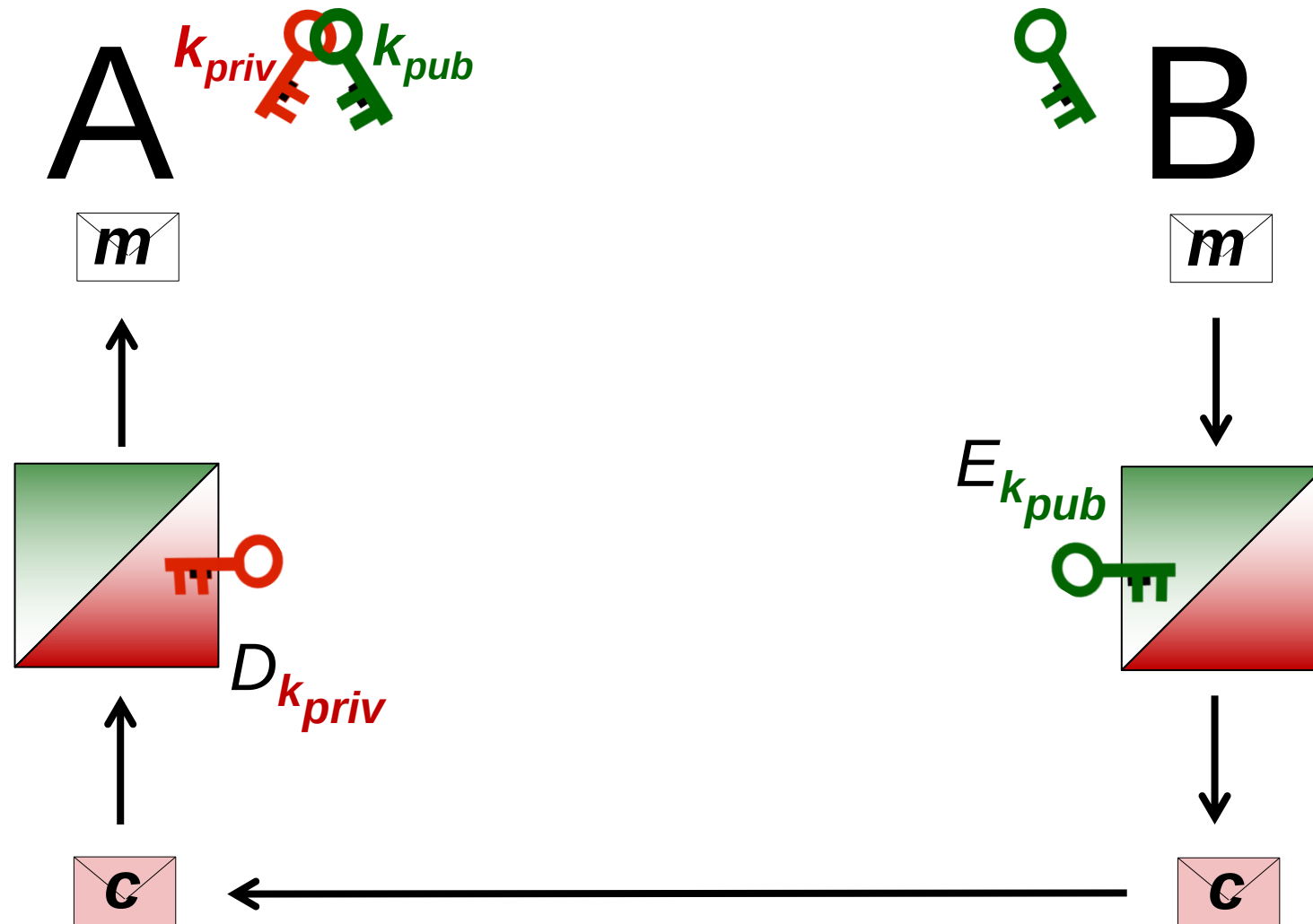
```

Faktorisierungs-Problem (Integer Factorization Problem, IF-Problem)

Gegeben sei eine natürliche Zahl $n \in \mathbb{N}$, die mindestens zwei große Primteiler p und q mit $p \neq q$ besitzt. Die Aufgabenstellung, allein aus der Kenntnis von n die Primteiler von n zu finden, wird als Faktorisierungs-Problem (IF-Problem) bezeichnet.

Bisher ist kein Algorithmus bekannt, der das IF-Problem effizient löst. Allerdings gibt es auch keinen Beweis, dass es einen solchen Algorithmus nicht geben kann.

3.2 Das RSA-Verschlüsselungsverfahren



3.2 Das RSA-Verschlüsselungsverfahren

RSA-Verschlüsselung

Bob möchte eine Nachricht m verschlüsselt an Alice schicken.

- Zunächst benötigt Bob den öffentlichen Schlüssel

$$k_{A, \text{pub}} = (n, e)$$

- Die zu verschlüsselnde Nachricht sei $m \in \mathbb{Z}_n$

- Bob berechnet die Geheimtextnachricht

$$c = E_{k_{A, \text{pub}}}(m) := m^e \bmod n \quad (3.5)$$

- Nach dem Empfang von c berechnet Alice mithilfe ihres privaten Schlüssels

$$m = D_{k_{A, \text{priv}}}(c) := c^d \bmod n \quad (3.6)$$

z.B. ein Schlüssel k_s
für symmetrisches Verfahren

in $\mathbb{Z}_n$:

$$c^d = (m^e)^d = m^{e \cdot d} = m^{1 + k \cdot \varphi} = m$$

Satz 3.1

Beispiel 3.3

Mit dem Schlüsselpaar aus dem „Schulbuch“-Beispiel 3.2

$$k_{A, pub} = (n, e) = (1763, 11)$$

$$k_{A, priv} = (n, d) = (1763, 611)$$

würde Bob beispielsweise die „Nachricht“ $m = 1234$ wie folgt verschlüsseln:

$$\begin{aligned}
 c &= 1234^{11} \bmod 1763 = 1234^{10} \cdot 1234 \bmod 1763 \\
 &= 1287^5 \cdot 1234 \bmod 1763 \\
 &= 1287^4 \cdot 1287 \cdot 1234 \bmod 1763 \\
 &= 912^2 \cdot 1458 \bmod 1763 \\
 &= 1371 \cdot 1458 \bmod 1763 \\
 &= 1439
 \end{aligned}$$

Beispiel 3.3

Mit ihrem privaten Schlüssel kann Alice c wieder entschlüsseln:

$$\begin{aligned}
 1439^{611} \bmod 1763 &= 1439^{610} \cdot 1439 \bmod 1763 \\
 &= 959^{305} \cdot 1439 \bmod 1763 \\
 &= 959^{304} \cdot 959 \cdot 1439 \bmod 1763 \\
 &= 1158^{152} \cdot 1335 \bmod 1763 \\
 &= 1084^{76} \cdot 1335 \bmod 1763 \\
 &= 898^{38} \cdot 1335 \bmod 1763 \\
 &= 713^{19} \cdot 1335 \bmod 1763 \\
 &= 713^{18} \cdot 713 \cdot 1335 \bmod 1763 \\
 &= 625^9 \cdot 1598 \bmod 1763 \\
 &= 625^8 \cdot 625 \cdot 1598 \bmod 1763 \\
 &= 1002^4 \cdot 892 \bmod 1763 \\
 &= 857^2 \cdot 892 \bmod 1763 \\
 &= 1041 \cdot 892 \bmod 1763 \\
 &= 1234 = m
 \end{aligned}$$

- **Schutz vor Brute-Force-Angriff bei kleiner Nachrichtenmenge**

$$m_1 \rightarrow \bar{E}_{k_{pus}}(m_1) = c_1$$

$$m_2 \rightarrow \bar{E}_{k_{pus}}(m_2) = c_2$$

⋮

$$\mathcal{M} = \{m_1, m_2, \dots, m_n\}$$

n klein

Angriffen steht c_i

Frage m_i ?

möglich per Brute-Force-Angriff

durch Berechnung von $\bar{E}(m_1), \bar{E}(m_2), \dots$

Beispiel 3.4 (Padding nach PKCS #1, Version 1)

$$EM = 0x00 \parallel 0x02 \parallel PS \parallel 0x00 \parallel M$$

```
1 Cipher rsaCipher = Cipher.getInstance("RSA/ECB/PKCS1Padding");  
  rsaCipher.init(Cipher.ENCRYPT_MODE, publicKey);  
  byte[] ciphertext = rsaCipher.doFinal(message);  
  
5 rsaCipher = Cipher.getInstance("RSA/ECB/NoPadding");  
  rsaCipher.init(Cipher.DECRYPT_MODE, privateKey);  
  System.out.println(Dump.dump(rsaCipher.doFinal(ciphertext)));
```

Abb. 3.2 RSA-Verschlüsselung mit PKCS1-v1_5-Padding und RSA-Entschlüsselung ohne Entfernung des Paddings

PS: enthält zufällige Bytes und $\neq 0x00$

Längstlänge von PS sind 8 Bytes

Beispiel 3.4 (Padding nach PKCS #1, Version 1)

$$EM = 0x00 \parallel 0x02 \parallel PS \parallel 0x00 \parallel M$$

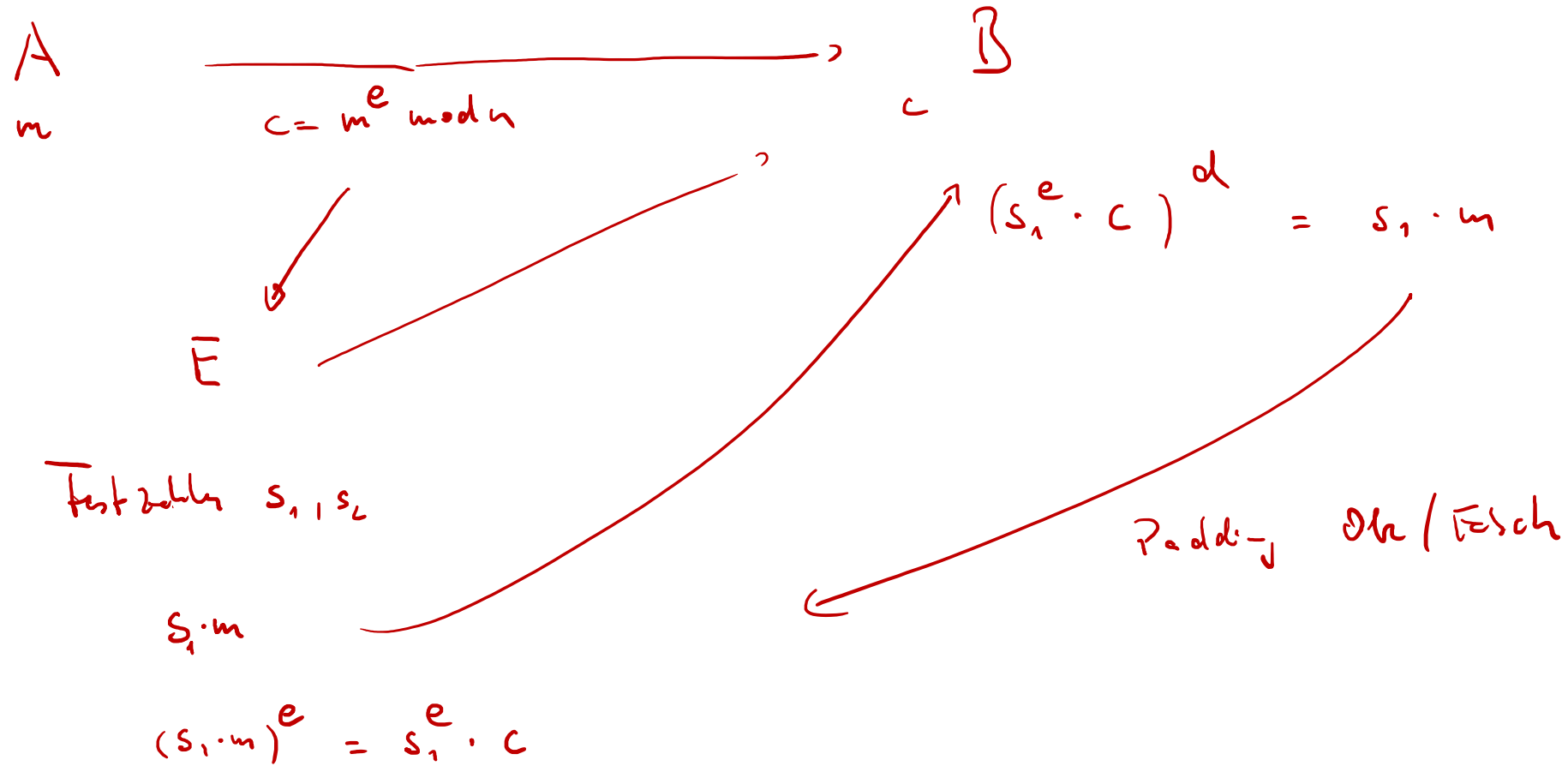
Beispiel 3.5 (*Bleichenbacher-Angriff*)

Im Jahr 1998 erregte die Veröffentlichung eines Angriffs auf RSA-verschlüsselte Nachrichten durch D. Bleichenbacher² große Aufmerksamkeit. Der *Bleichenbacher-Angriff* ist ein adaptiver Chosen-Ciphertext-Angriff auf RSA-verschlüsselte PKCS1-v1_5-kodierte Nachrichten.

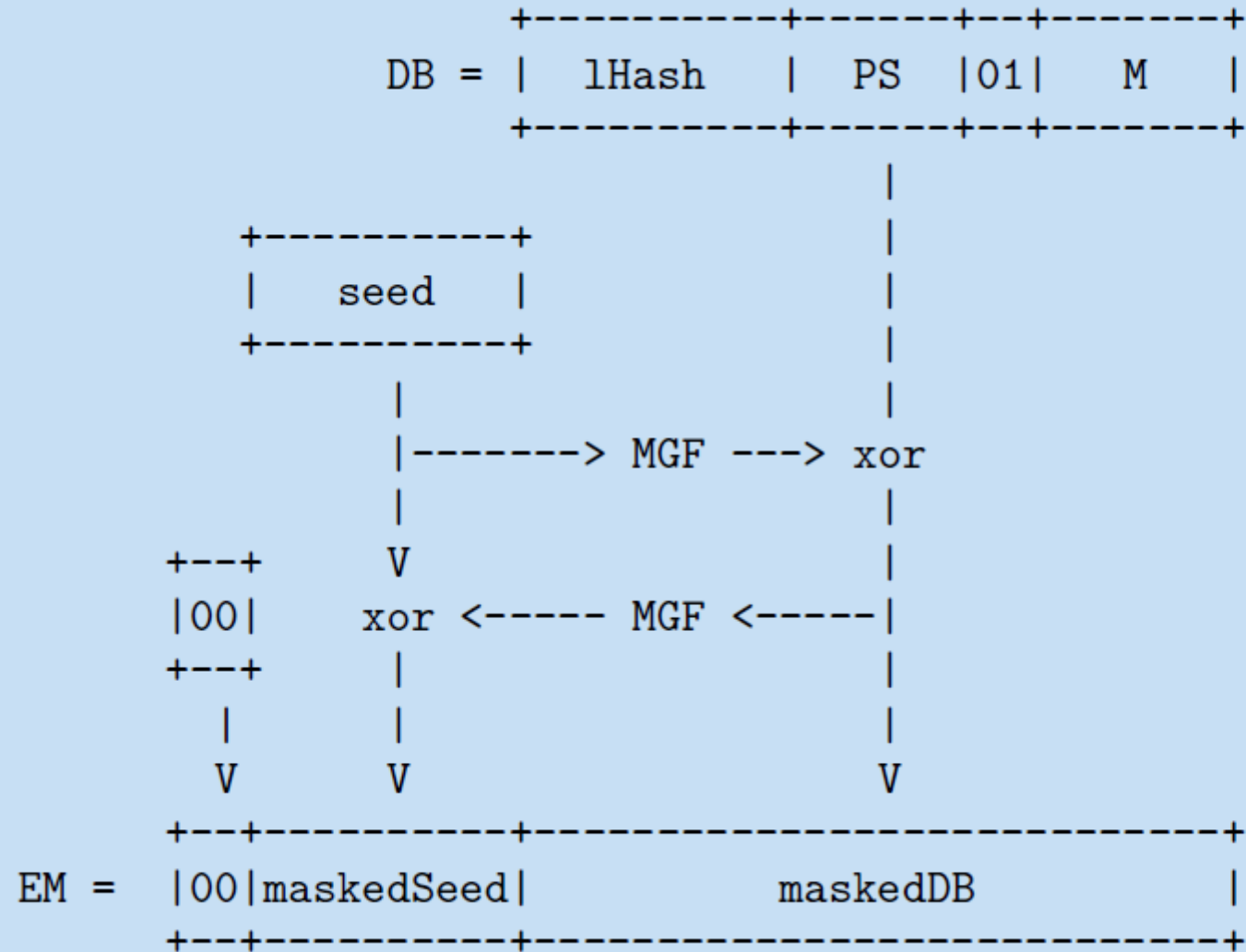
- Chosen ciphertext attacks against protocols based on the RSA encryption standard PKCS #1, [Advances in Cryptology — CRYPTO '98](#)
- [The ROBOT Attack - Return Of Bleichenbacher's Oracle Threat](#)

Beispiel 3.4 (Padding nach PKCS #1, Version 1)

$$EM = 0x00 \parallel 0x02 \parallel PS \parallel 0x00 \parallel M$$



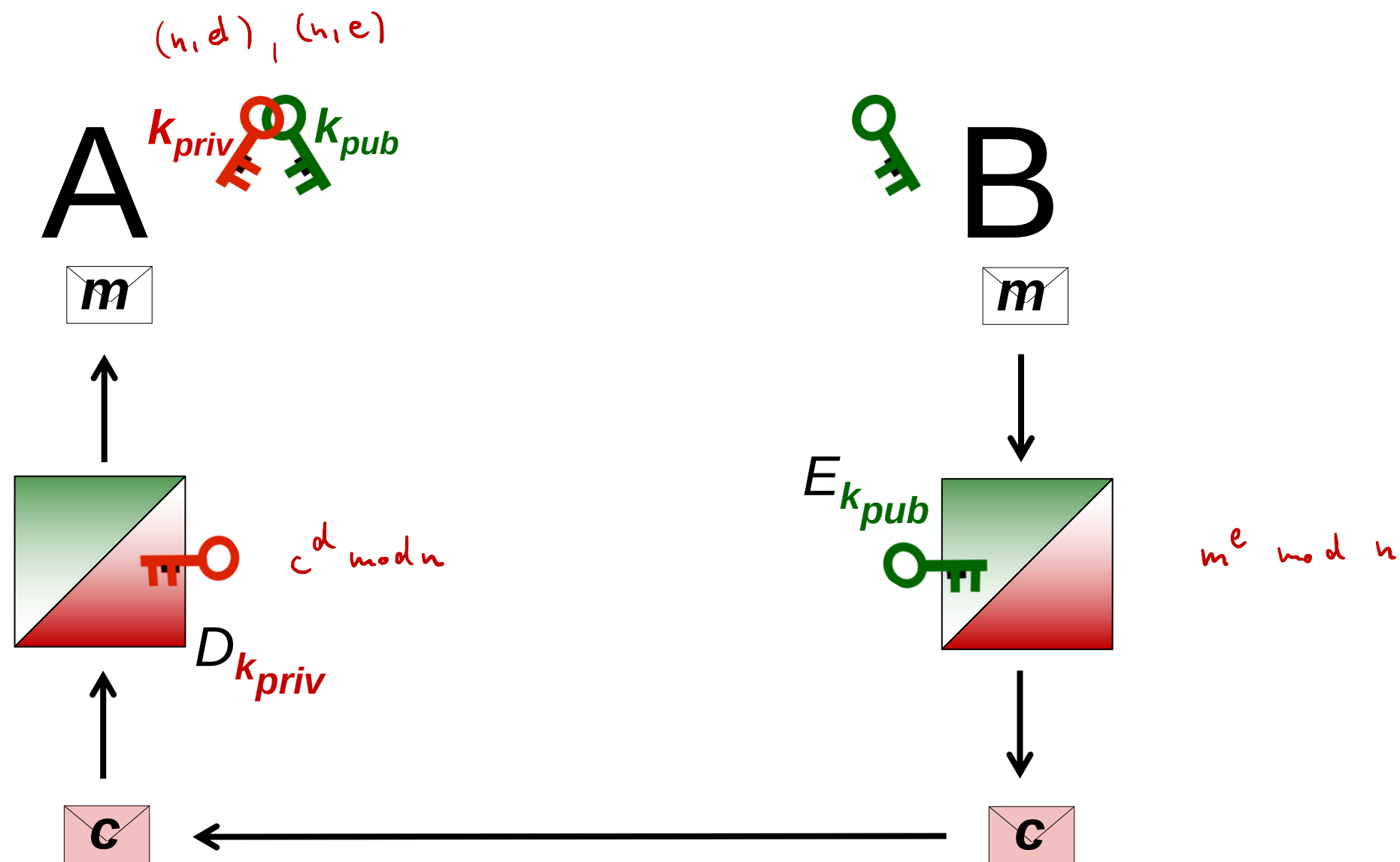
Beispiel 3.6 (Optimal Asymmetric Encryption Padding (OAEP))



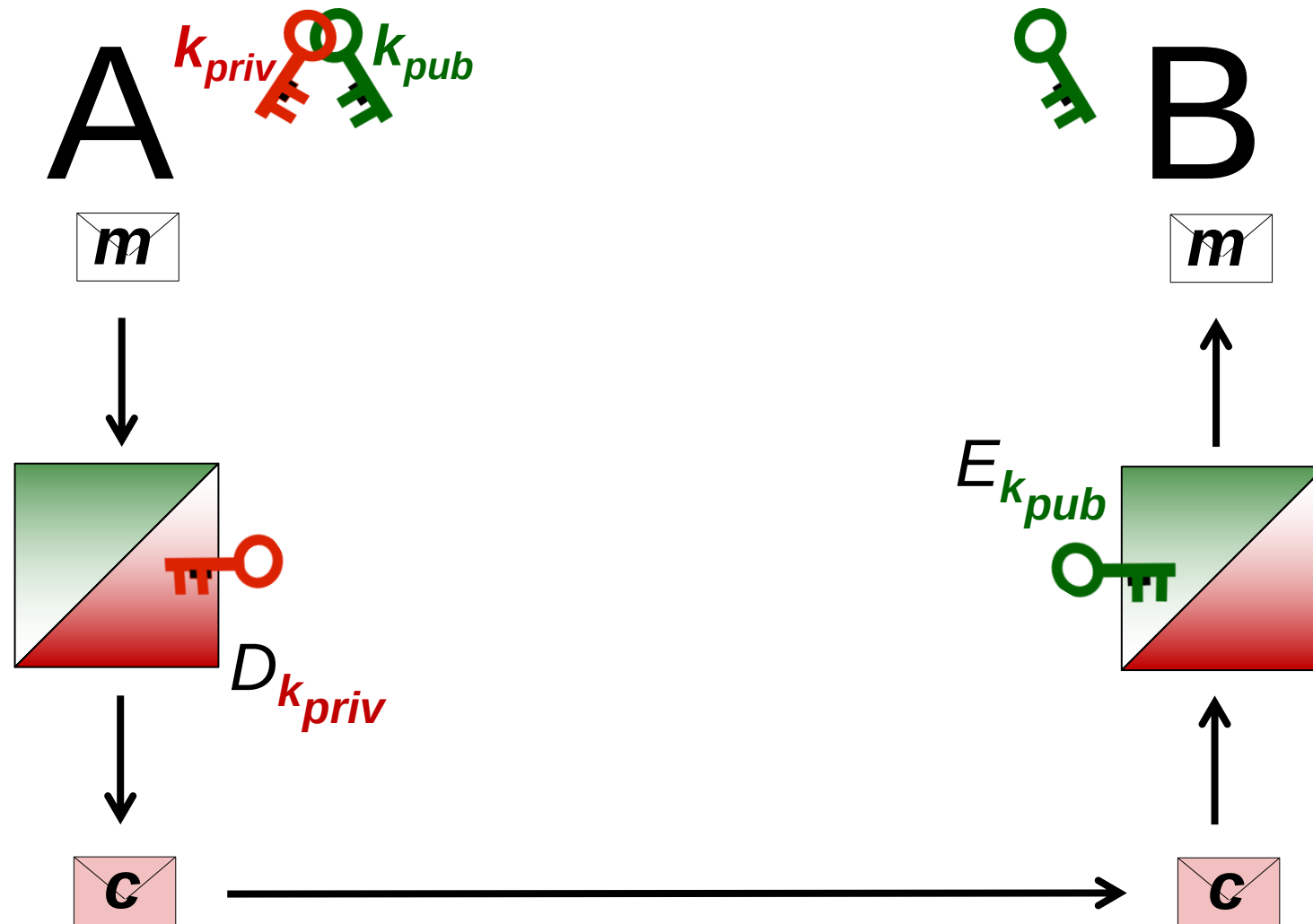
Die von *MGF1* erzeugten pseudozufälligen Bytes sind:

$$h(\text{Seed} || 00000000) || h(\text{Seed} || 00000001) || h(\text{Seed} || 00000002) || \dots$$

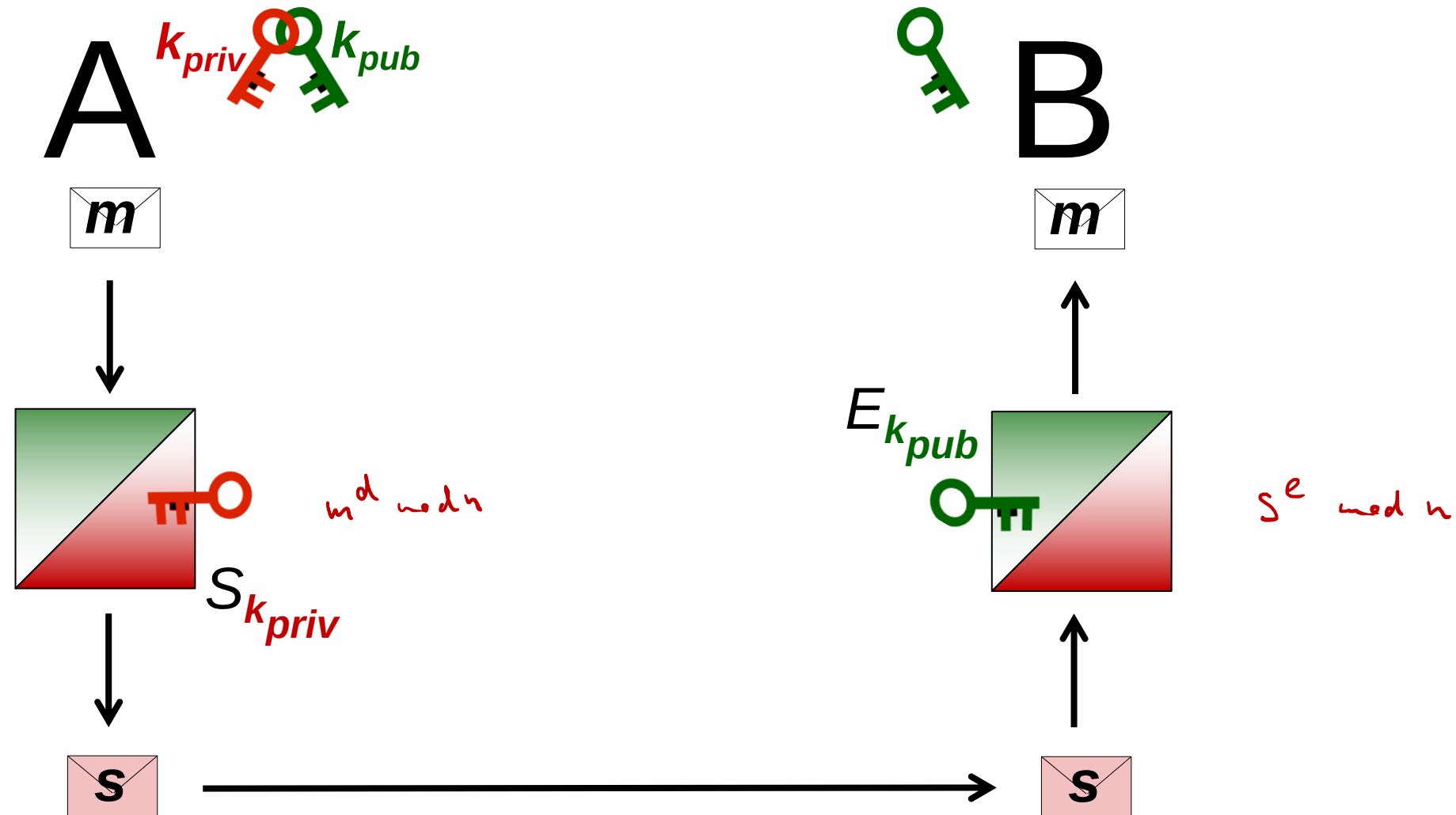
3.2 Das RSA-Verschlüsselungsverfahren



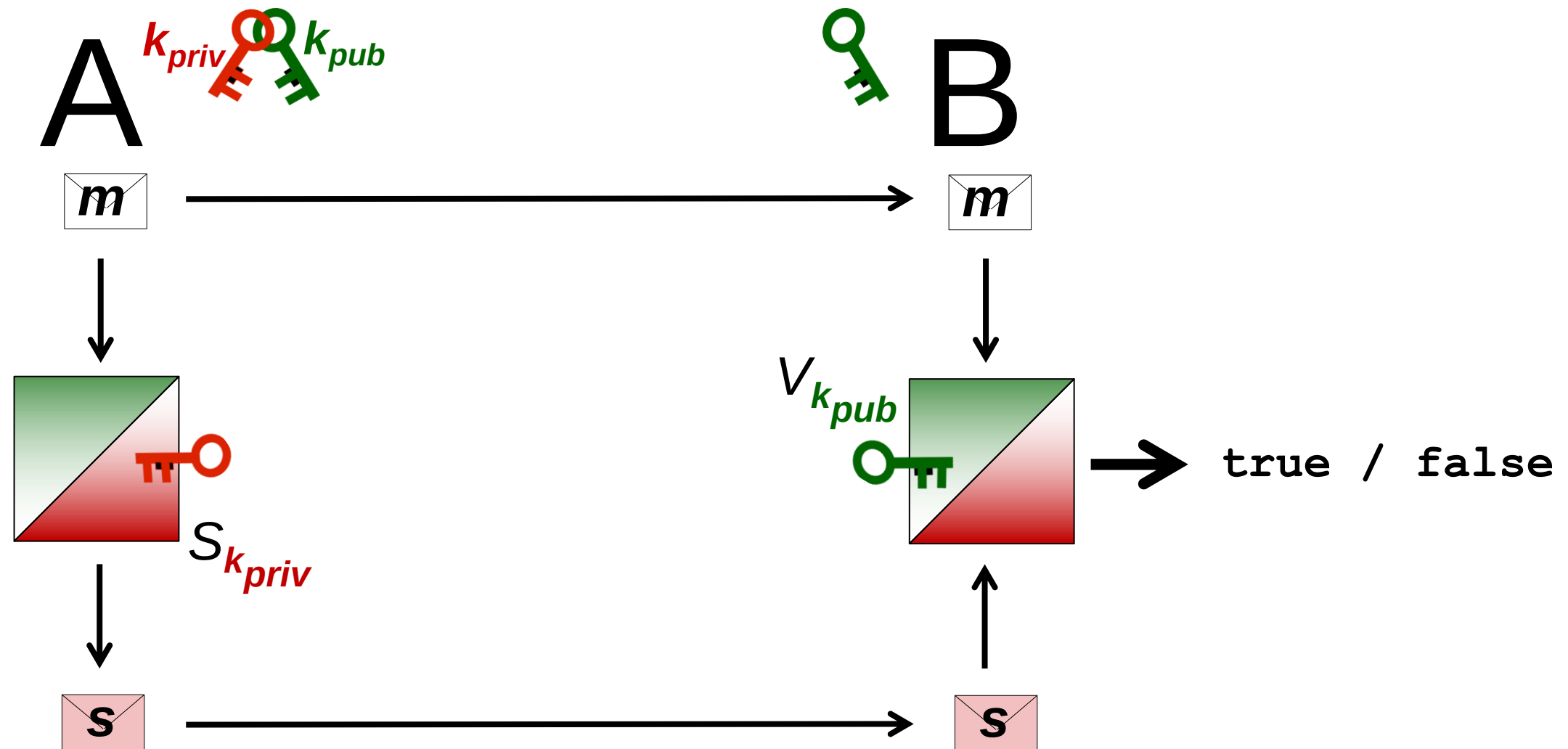
3.3 Das RSA-Signaturverfahren



3.3 Das RSA-Signaturverfahren



3.3 Das RSA-Signaturverfahren



RSA-Signaturverfahren

Alice möchte eine Nachricht m an Bob schicken, sodass sich Bob von der Authentizität von m überzeugen kann, und Bob die Authentizität von m gegebenenfalls auch Dritten gegenüber nachweisen kann.

$$m \in \mathbb{Z}_n$$

- Zunächst benötigt Bob den öffentlichen Schlüssel

$$k_{A, \text{pub}} = (n, e)$$

- Alice berechnet eine Signatur für m entsprechend

$$s = S_{k_{A, \text{pri}}}(m) := m^d \bmod n$$

- Nach dem Empfang von m und s berechnet Bob mithilfe des öffentlichen Schlüssels von Alice die Zahl

$$\tilde{m} = s^e \bmod n$$

und überprüft die Gültigkeit von

$$\tilde{m} = m \quad)$$

Beispiel 3.7 (Signature Scheme with Appendix RSASSA-PKCS1-v1_5)

- Mit einem Hashverfahren *Hash* wird der Hashwert

$$h = \text{Hash}(m)$$

der Nachricht m berechnet.

- Der Hashwert h wird zu einer Zahl

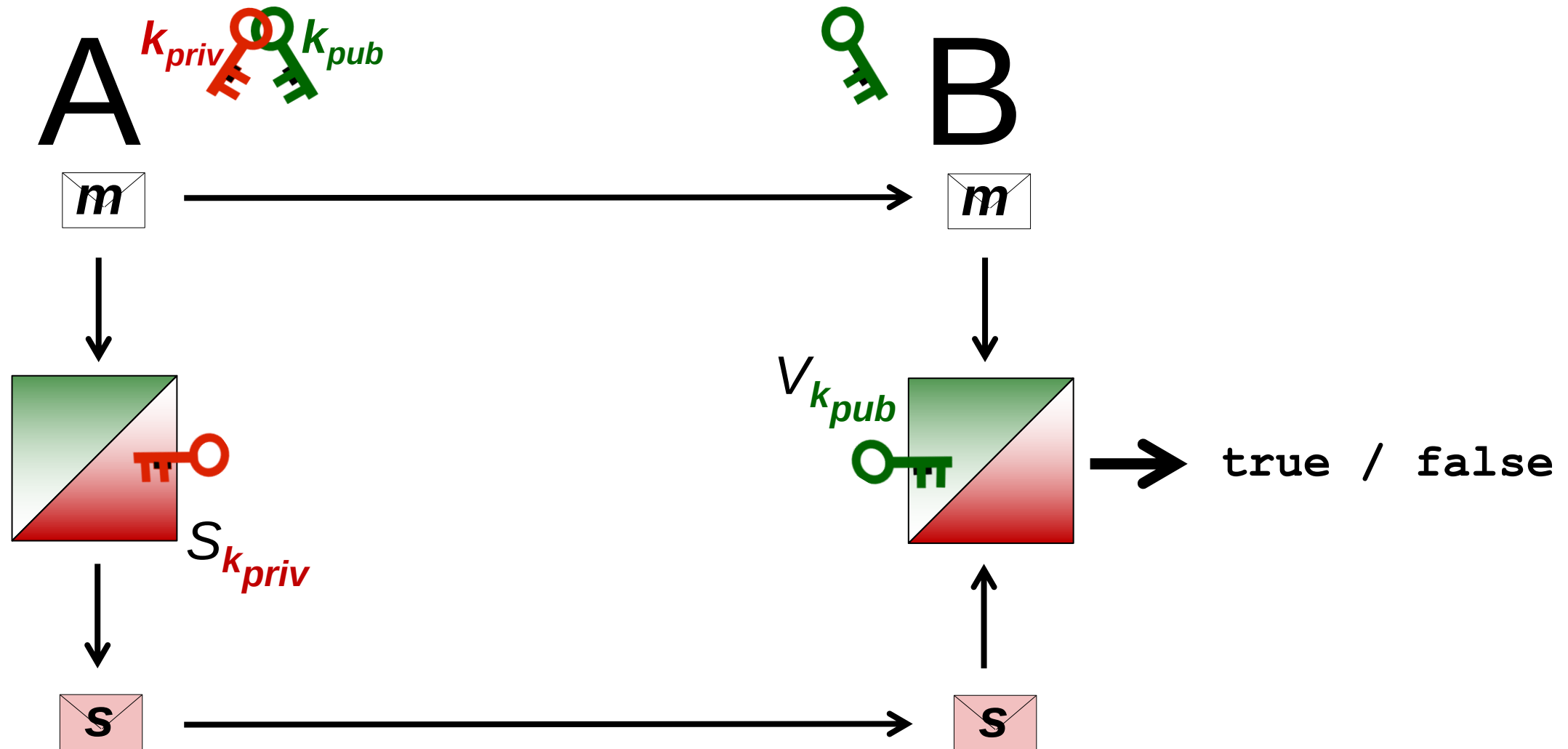
$$\tilde{h} < n$$

kodiert.

- Auf $\tilde{h}$ wird die private RSA-Operation angewandt. Die Signatur der Nachricht ist:

$$s = \tilde{h}^d \bmod n$$

3.3 Das RSA-Signaturverfahren



3.3 Das RSA-Signaturverfahren

